UNIVERSITÀ DEGLI STUDI DI MILANO BICOCCA

Autoencoders

Lecture 17

Course of:

Statistics and data analysis

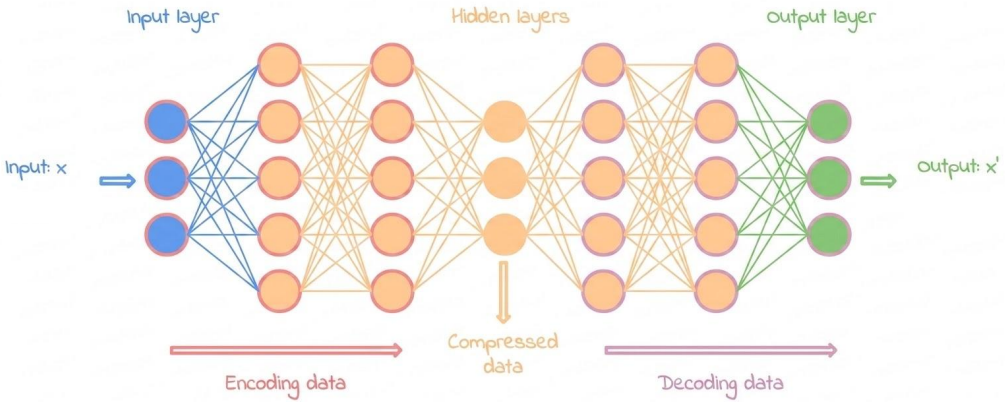
Federico De Guio - Martino Borsato

De-noising and **anomaly detection**
with a semi-supervised approach

The **AutoEncoder** (AE)

- Feed forward neural network with **same input and output shapes**
- Goal: build a model that identifies **fundamental characteristics in the input data**
  - Reconstruct original input by minimizing mean-squared error loss (or similar)

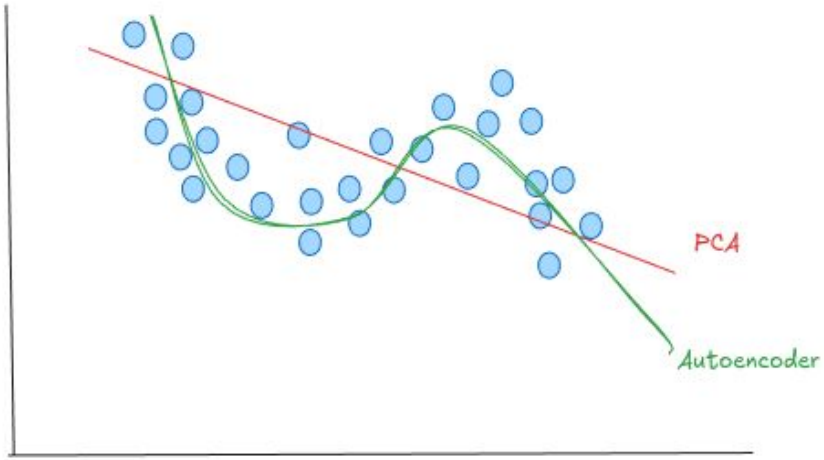
- Combine an **encoder** that converts input data in a different representation, with a **decoder** that converts the new representation back to the original input

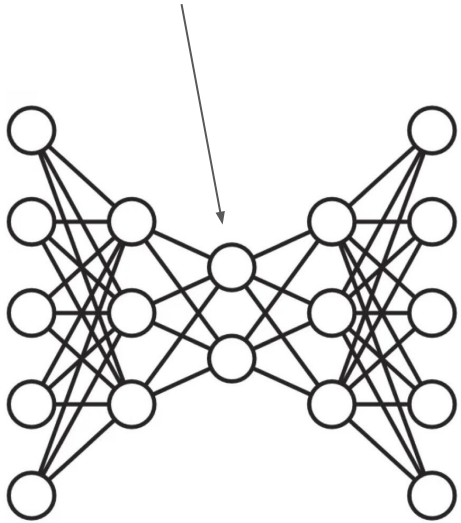


Autoencoders vs PCA

- A way to describe it is that **the autoencoder learns a nonlinear lower-dimensional manifold** from the training dataset
- The **PCA attempts to discover a lower dimensional hyperplane**
- In both cases **dimensionality reduction** is performed

- Usually autoencoders are **undercomplete** (bottleneck layer performs data compression)





A **self-supervised training** approach

- The AE learns what to store in the latent space which is needed to "**reconstruct well**" the input events
  - The AE compresses the input sample and knows how to reconstruct it from the bottleneck so that the distance with respect to the input is minimized (MSE)
- For AEs the training is often called: **Classification without labels**
  - Robust approach: don't need to know all the labels or the classes!

Simple implementation in keras

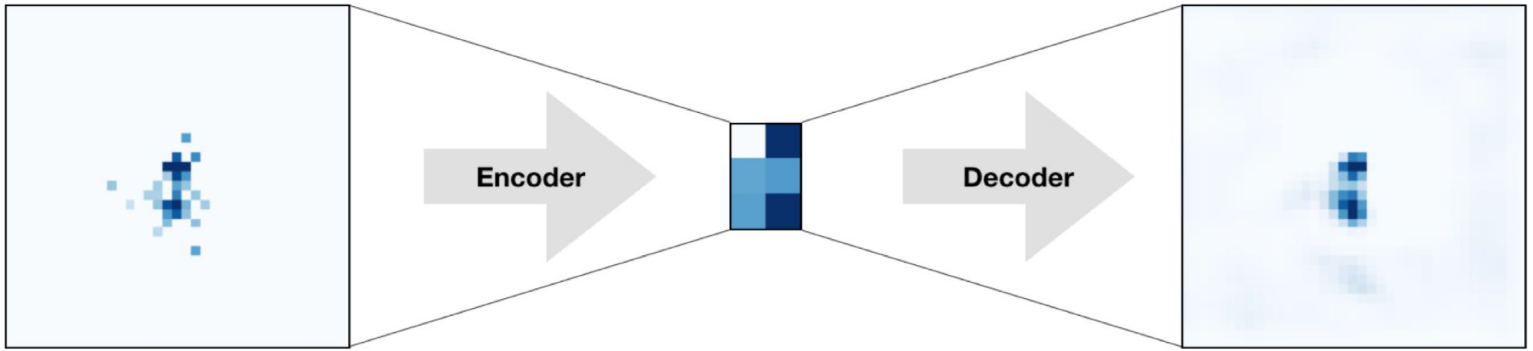
```
m = Sequential()
m.add(Dense(1000, activation='relu',
            input_shape=(784,)))
m.add(Dense(500, activation='relu'))
m.add(Dense(250, activation='relu'))
m.add(Dense(32, activation='relu'))
m.add(Dense(2, activation='linear',
            name="bottleneck"))
m.add(Dense(32, activation='relu'))
m.add(Dense(250, activation='relu'))
m.add(Dense(500, activation='relu'))
m.add(Dense(1000, activation='relu'))
m.add(Dense(784, activation='sigmoid'))
```



Application: **outlier (anomaly) detection**

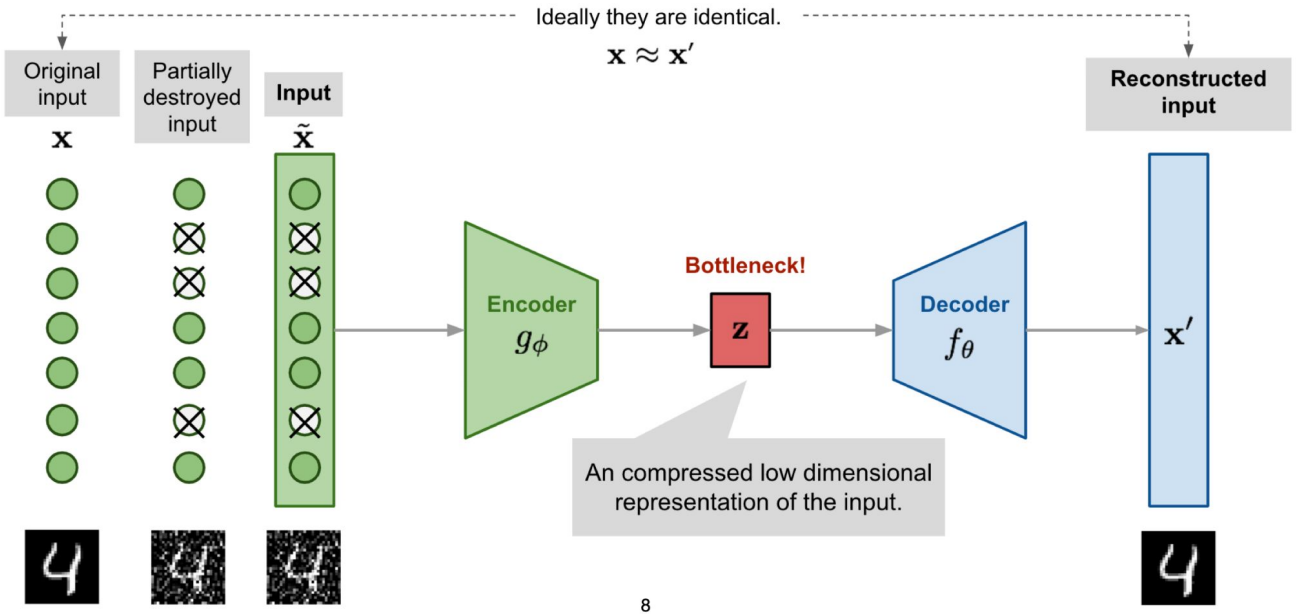
- The training dataset contains only one class, no outliers nor anomalies
- → **The model learns to reconstruct well the standard data** (low MSE) **while anomalies and outliers are poorly reconstructed** (high MSE)
  - One class classification

- Example: the AE is trained on the background events only → low MSE on bkg after reconstruction
  - If *x* is far from *AE(x) = Decoder(Encoder(x))*, then *x* has low probability to be background
  - → potentially indicate the presence of an **interesting anomaly (signal)**



Application: **de-noising model**

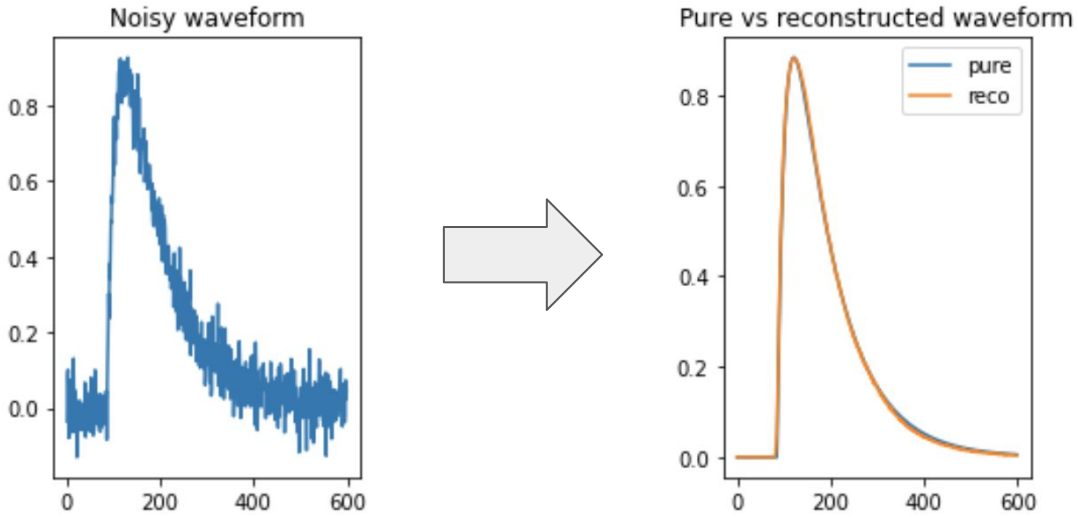
- Denoising autoencoders (Vincent et al. 2008) corrupt the input by **adding noise or masking values stochastically and then the model is trained to recover the original input** (not the corrupted one)
  - During training give noisy pulses in input to the AE and the pure pulses as the target
  - The **AE learns the relevant features** of the pulse and discards the noise



Examples…

De-noising of 1D pulses: definition of the problem

- Use an available pulse library and train a DNN to remove the noise from the pulses
- What are the main parameters of a pulse like this?



- Example available here
- Pulse library available here

Additional information and possible next steps

- Out of curiosity, this is how the training dataset is created
  - Simplistic MC simulation of scintillation light + SiPM response
  - Basically the pulses are the convolution of two exponentials
  - Parameters are:
    - Tau raise
    - Tau decay
    - t0 shift
    - amplitude

- **How many parameters are needed in the bottleneck of the AE?**
- Does a **convolutional autoencoder** make sense for this application?
  - Try to replace dense layers with Conv1D layers. Does the overall performance improve?

Variational Autoencoders

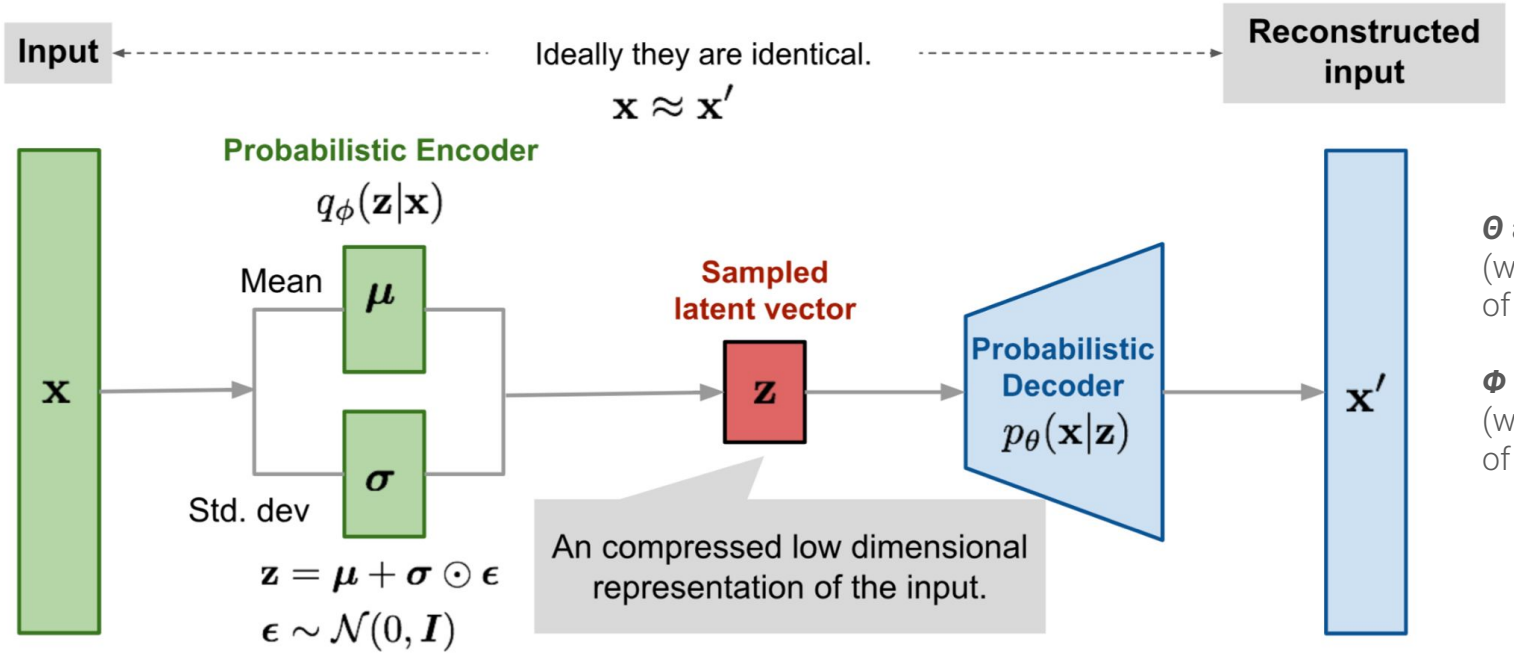
- Prince, cap. 17

Variational autoencoders (**VAE**)

- Variational autoencoder or VAEs (Kingma & Welling, 2014) **rooted in variational Bayesian methods**
- Instead of mapping the input into a fixed vector, we **map it into a distribution p_θ** parameterized by θ

Input data x and encoding vector z are related by:

- **Prior** $p_\theta(z)$
- **Likelihood** $p_\theta(x \mid z)$
- **Posterior** $q_\phi(z \mid x)$



Θ are the parameters (weights and biases) of the decoder

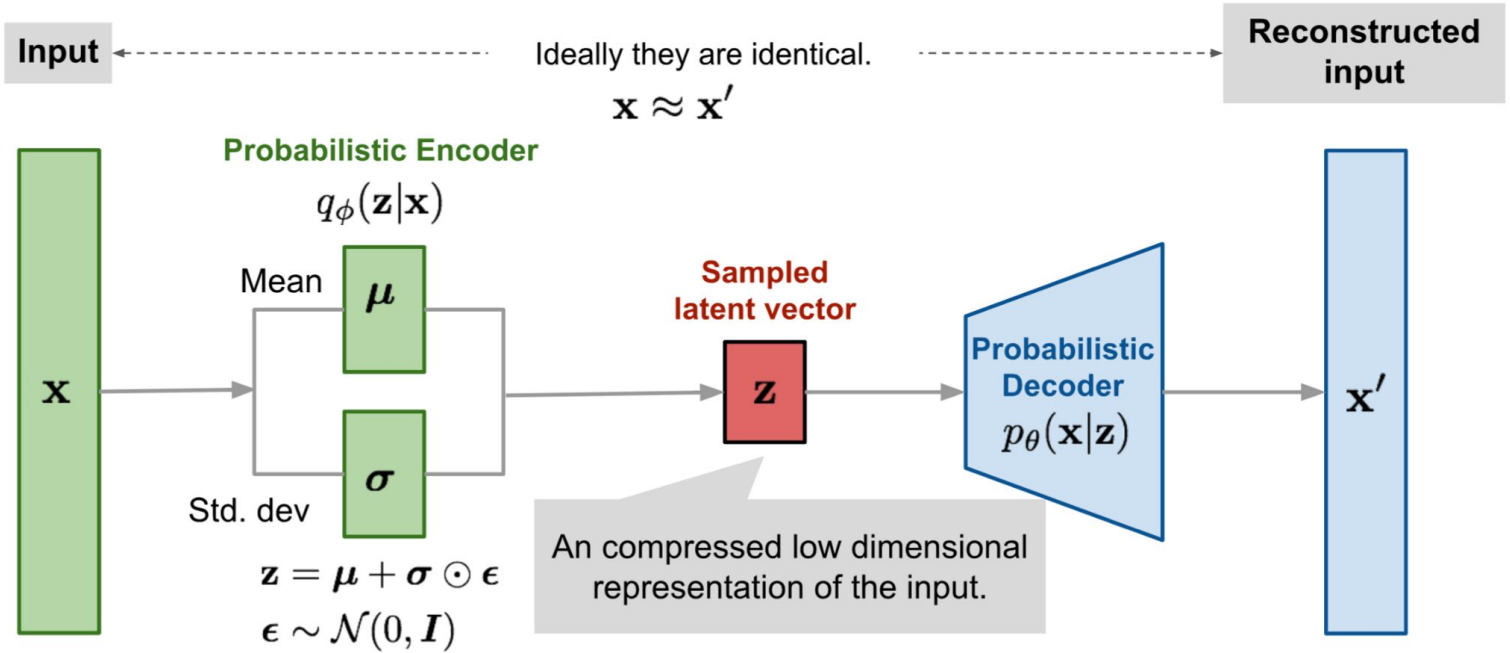
Φ are the parameters (weights and biases) of the encoder

Likelihood $p_\theta(x \mid z)$

- Given the latent space *z*, **what is the probability to observe a specific pulse *x*?**
  - During training, when computing the reconstruction loss, I'm trying to maximize this term

Input data x and encoding vector z are related by:

- **Prior** $p_\theta(z)$
- **Likelihood** $p_\theta(x \mid z)$
- **Posterior** $q_\phi(z \mid x)$

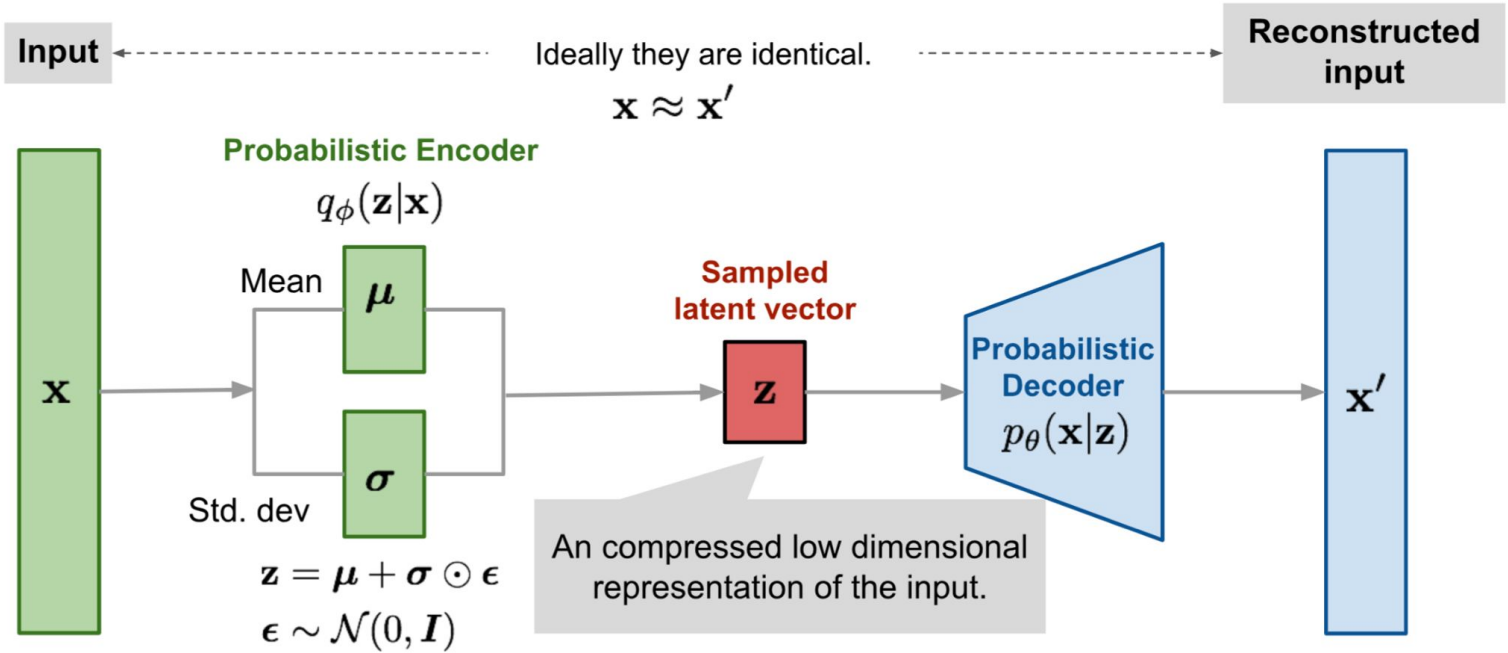


Prior $p(z)$

- **Constraint on the shape of the latent space**
- For standard VAEs, the prior is a standard Gaussian *N(0, I)*
  - We are forcing the parameters of the latent space to follow this simple distribution

Input data x and encoding vector z are related by:

- **Prior** $p_\theta(z)$
- **Likelihood** $p_\theta(x \mid z)$
- **Posterior** $q_\phi(z \mid x)$

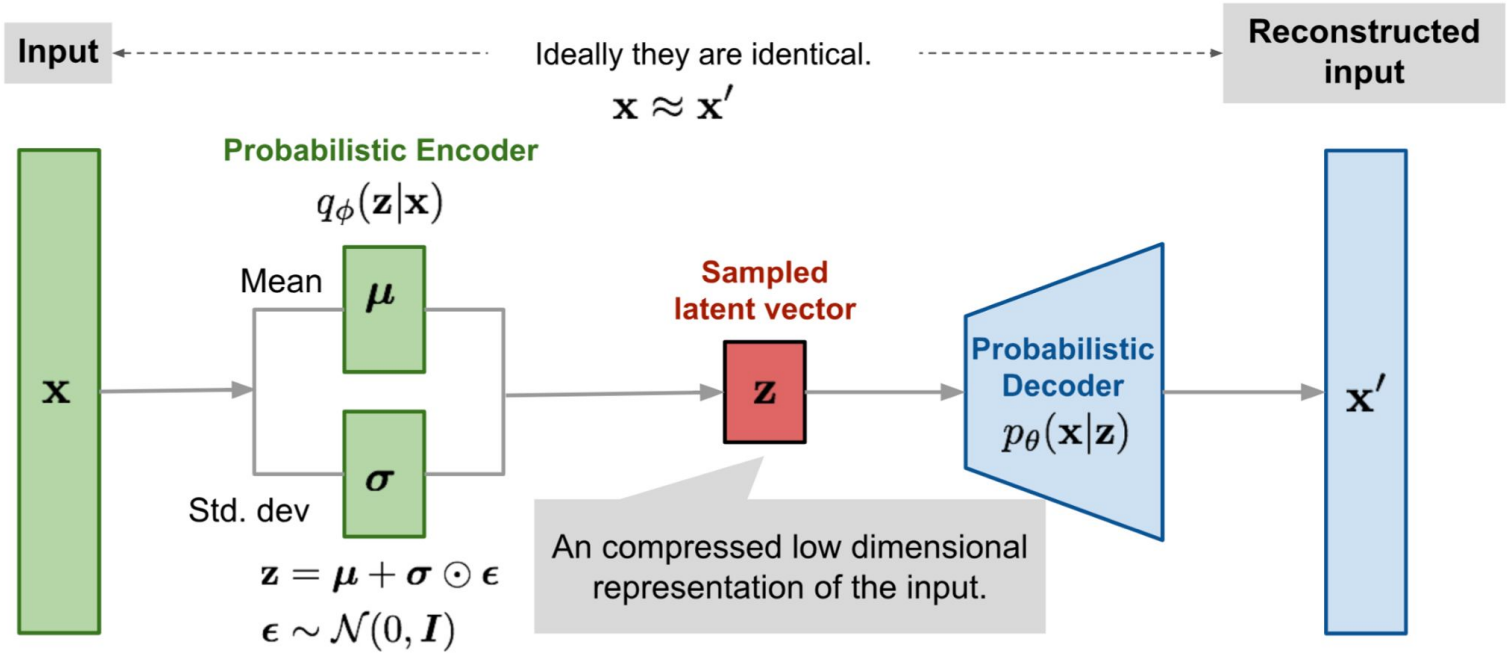


Posterior $q_\phi(z|x)$ - approximated

- **Given the pulse *x*, what are the parameters *z* that generated it?**
- It is called **variational posterior** because it's an approximation
  - The real posterior *p(z|x)* is mathematically inaccessible

Input data x and encoding vector z are related by:

- **Prior** $p_\theta(z)$
- **Likelihood** $p_\theta(x \mid z)$
- **Posterior** $q_\phi(z \mid x)$



In short

- With VAEs we are not just trying to compress the data, but to **solve a statistical problem**

- Given θ, we want to **generate a sample like a real data point** by following:
  - Sample $z^{(i)}$ from a prior distribution $p_\theta(z)$
  - Generate a value $x^{(i)}$ from the conditional distribution $p_\theta(x \mid z = z^{(i)})$
  - (once training is done, I'm using the decoder only here)

- In order to train the model, we look for the optimal parameter θ^* of the network that **maximizes the probability of generating real data samples (evidence)** or minimizes the loss:

$$\theta^* = \arg\min_\theta \left(-\sum_{i=1}^{n} \log p_\theta(\mathbf{x}^{(i)}) \right)$$

log-likelihood

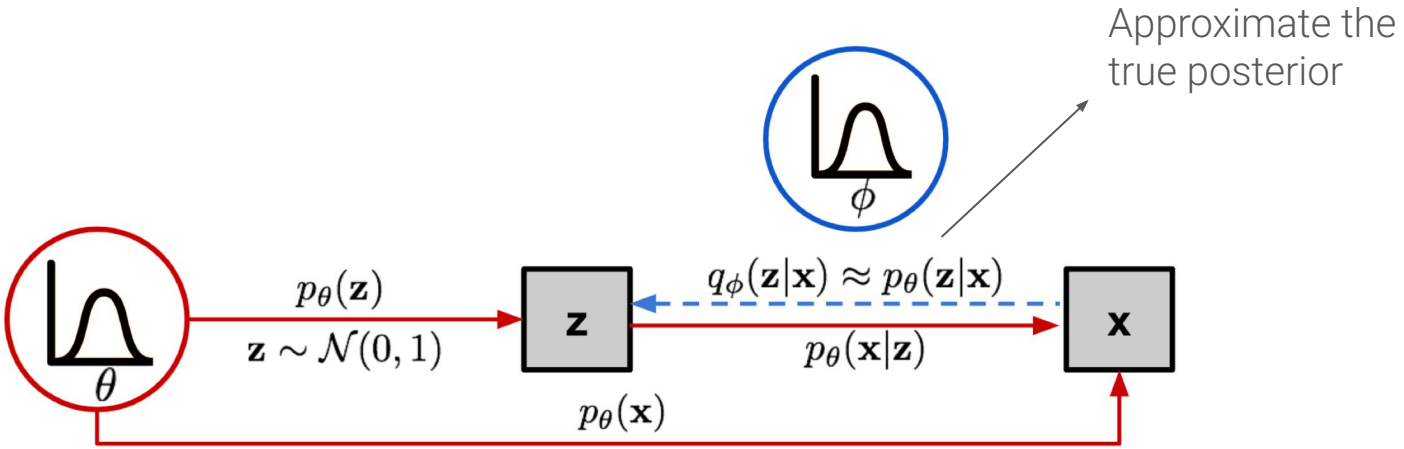
where

$$p_\theta(\mathbf{x}^{(i)}) = \int p_\theta(\mathbf{x}^{(i)} \mid \mathbf{z}) p_\theta(\mathbf{z}) d\mathbf{z}$$

The idea behind the **variational method**

- However it is not easy to compute $p_\theta(x^{(i)})$ **because of integration over all values of *z***
- → we introduce an **approximation function**:
  - Give *x* to the encoder $q_\phi(z \mid x)$ and ask what *z* are more probable for a specific input
    - During training, we will find the normal distribution that is "closest" to the true posterior
  - The encoder **narrows down the space** and tells us what parameters to pick from the latent space for that input

- $p_\theta(x|z)$ defines a generative model called **probabilistic decoder**
- $q_\phi(z \mid x)$ is the approximation function called **probabilistic encoder**



Variational inference and ELBO

- Goal: estimate $q_\phi(z \mid x)$ that best approximate the posterior by minimising the KL (Kullback-Leibler) divergence:

$$D_{\mathrm{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) \| p_\theta(\mathbf{z}|\mathbf{x})) = \int q_\phi(\mathbf{z}|\mathbf{x}) \log \frac{q_\phi(\mathbf{z}|\mathbf{x})}{p_\theta(\mathbf{z}|\mathbf{x})} dz$$

→ KL quantifies the distance between posterior and true posterior

- In practice as directly minimising the KL is also not tractable because of $p_\theta(z|x)$, the **ELBO** (Evidence Lower Bound) is used as its tractable surrogate

$$L = \underbrace{-\log p_\theta(\mathbf{x})}_{\text{Pushes towards the true posterior (physical accuracy)}} + \underbrace{D_{\mathrm{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) \| p_\theta(\mathbf{z}))}_{\text{Pushes towards a regular prior (continuous latent space)}} = -\ ELBO$$

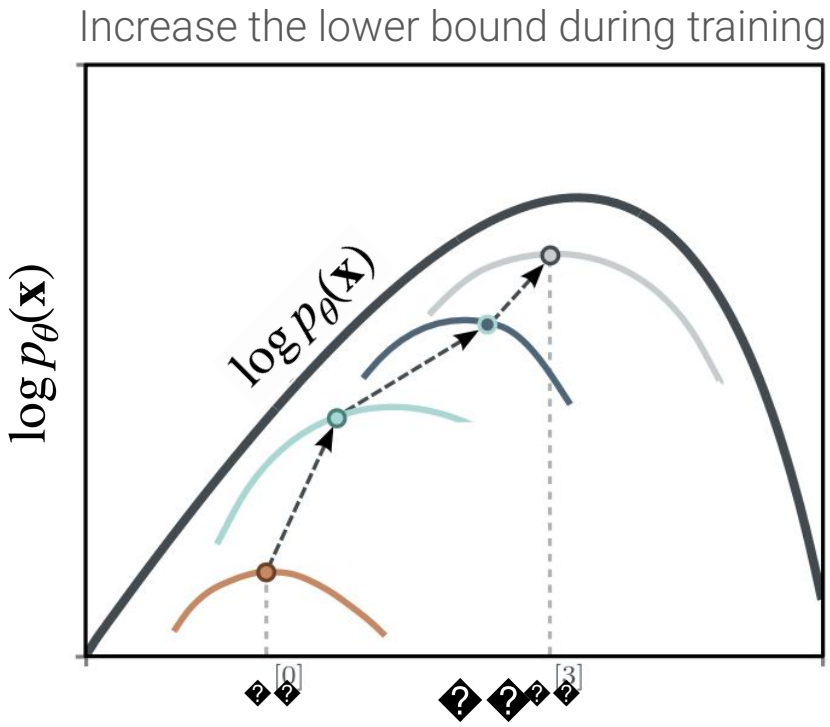
- By minimizing the loss, we are **maximizing the lower bound of the probability of generating real data samples**
  - re-frames the computation of an integral (the marginal likelihood from exact inference) as the optimization of its lower bound

How it works visually

- During training the VAE updates **both the parameters ϕ of the encoder and the parameters θ of the decoder to increase the lower bound**

- ELBO is the cost function and it is composed by two terms that try to balance with each other:
  - **The reconstruction loss** → looks at the quality of the reconstruction
  - The KL divergence is a **complexity penalty** → forces the latent space z to be gaussian and centered in 0

- Ideal training:
  - Good reconstruction quality
  - Regular latent space to be used as a probabilistic model



Reparametrization trick

Original form:

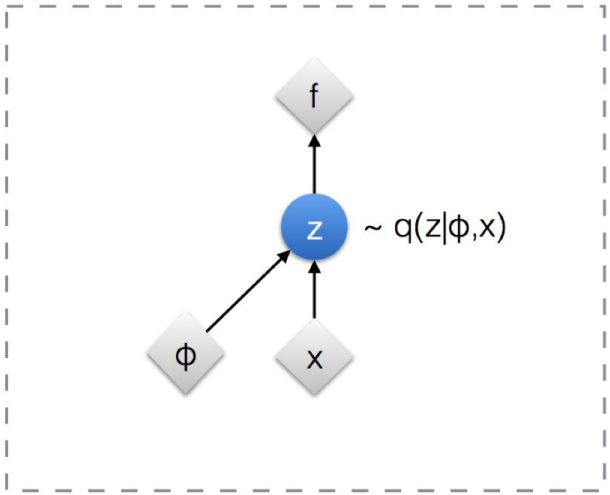
$$\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x}^{(i)}) = \mathcal{N}(\mathbf{z}; \boldsymbol{\mu}^{(i)}, \boldsymbol{\sigma}^{2(i)}\boldsymbol{I})$$

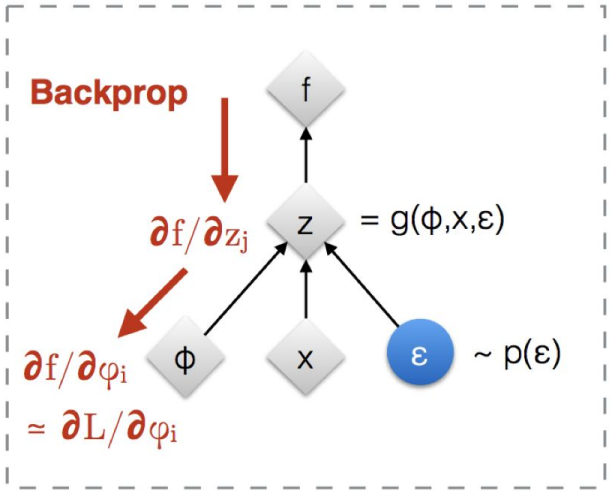
$\Leftrightarrow$

Reparametrized form:

$$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}, \text{ where } \boldsymbol{\epsilon} \sim \mathcal{N}(0, \boldsymbol{I})$$

During training can't compute the derivative of a random node





During training ε is considered an external constant

: Deterministic node

: Random node

[Kingma, 2013]
[Bengio, 2013]
[Kingma and Welling 2014]
[Rezende et al 2014]

Considerations

- When performing inference, I can consider my VAE is an **instantaneous Bayesian fitter**
  - For each example, **the encoder receives the input and estimates the parameters mu and sigma** → it is effectively returning a posterior distribution while the frequentist approach would return a single value
  - With respect to a MCMC based bayesian fit, the exploration of the posterior happened during training → the model has learned how to perform the fit
  - The physics of the phenomenon is learned during training
  - **Caveat**: the fitter is bound to what it has seen during training

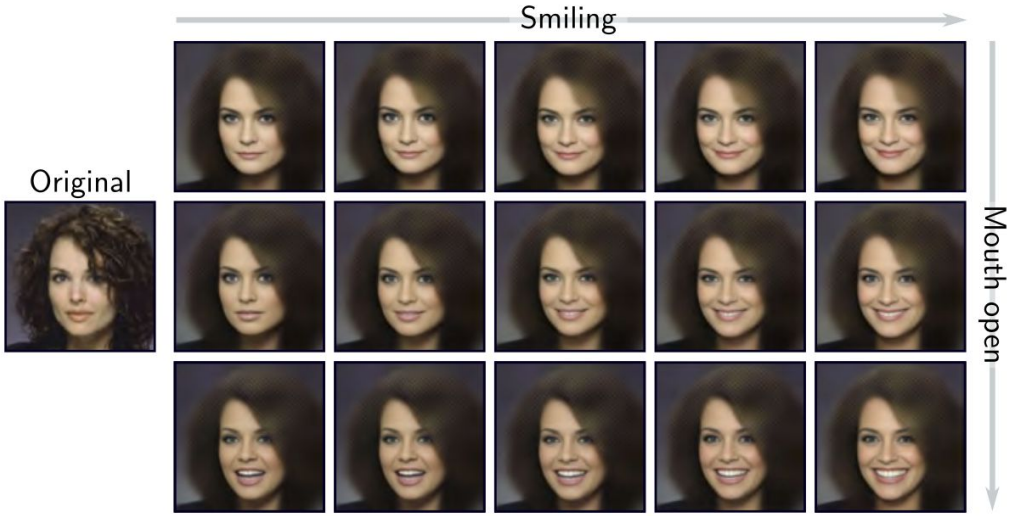
- What happens if ε = 0? → deterministic AE
- What happens if I set the KL loss to 0? → deterministic AE
- The mechanics of the AE remains the same, but mathematically it matches the deterministic AE since there is **no constraint on the shape of the posterior** during training

VAEs as generative models

- Project the original image to the latent space
- Then **manipulate the latent space** and use the decoder to reconstruct the sample
  - In this example assuming that the two features I'm inspecting are *smiling* and *mouth open*

- **If I "turn ε on" I have created a MC generator**
  - Extract a random number from a standard gaussian and give it as input to the decoder
  - My model generates events according to the process it learned and each event will differ from the others

Deterministic inspection of the latent space

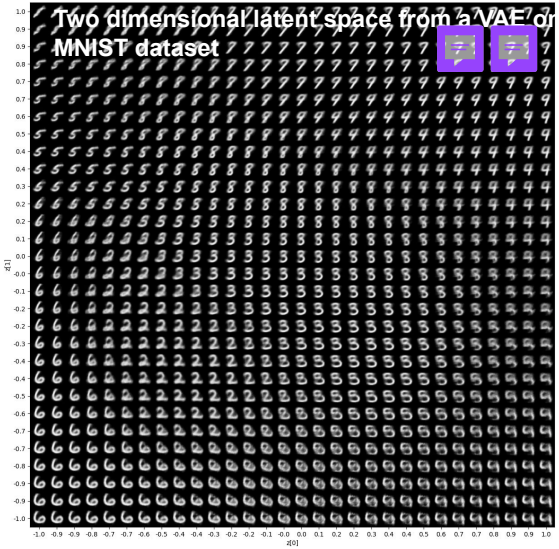


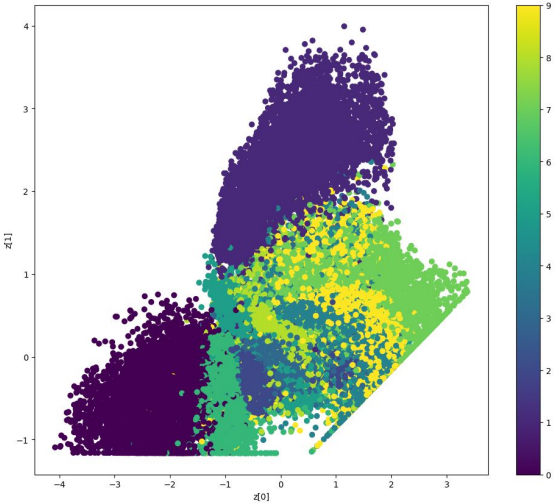
Interpretability for analysis

- Another added value of a regular latent space, is **interpretability**

- Physically Meaningful Manifold (**Disentanglement**)
  - The KL divergence forces the model to map independent physical factors (e.g., pulse shift vs. decay time) onto separate, orthogonal axes
- Continuous Feature Trajectories (**Smooth Interpolation**)
  - Unlike standard Autoencoders, where the space is fragmented, a VAE ensures a dense and continuous manifold. This enables a **smooth latent space scan**
- **Robustness to Noise** and Outlier Detection:
  - By forcing the encoder to produce distributions (μ,σ) rather than fixed points, the model learns to ignore high-frequency noise
  - The distance from the center of the prior *(N(0,I))* serves as a natural metric for anomaly detection

https://keras.io/examples/generative/vae/

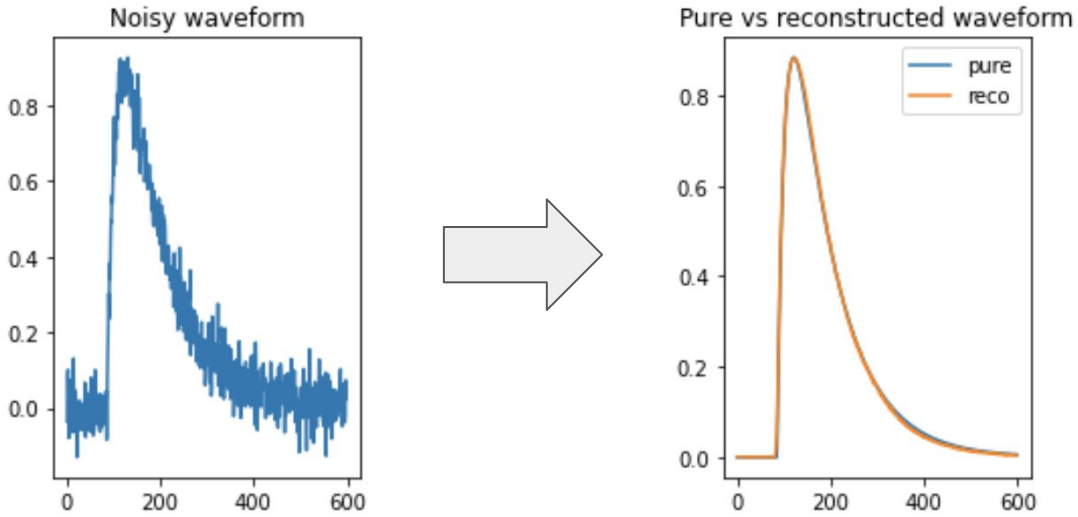




Examples…

De-noising of 1D pulses: let's use VAE this time

- Use an available pulse library and train a DNN to remove the noise from the pulses
- What are the main parameters of a pulse like this?



- Example available here
- Pulse library available here

Question:

Can VAE be used to quantify the uncertainty on their output?